Graph Traversal

To solve problems with graphs, you must know how to traverse (move through) them. Traversal in graphs is similar to trees — start at a vertex, visit neighbors, then neighbors of neighbors, and so on. But graphs are different from trees in two important ways:

- Cycles: A graph can have cycles. If you aren't careful, you could get stuck visiting the same vertices again and again (infinite loop).
- Disconnected graphs: A graph may have separate parts. Starting from one vertex will only reach its connected part, not the whole graph.

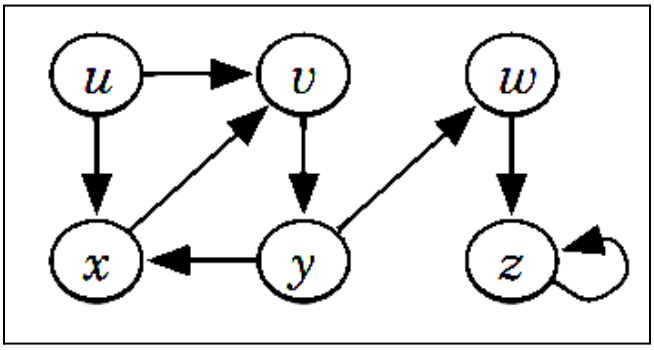
Because of these, simple recursion like in trees won't work. Instead, we use:

- A Stack or a Queue
- A visited list to track which vertices we have already seen.

BFS (Breadth-First Search)

- **Uses a queue** (First In, First Out).
- **Visits level by level**: Start from a vertex, visit all its neighbors first, then the neighbors' neighbors.
- **Steps**:
    1. Insert the starting vertex into the **queue** and mark it visited.
    2. While the queue is not empty
        - Remove a vertex from the front.
        - Visit its unvisited neighbors and add them to the queue.
- Good for finding the shortest path in an unweighted graph.

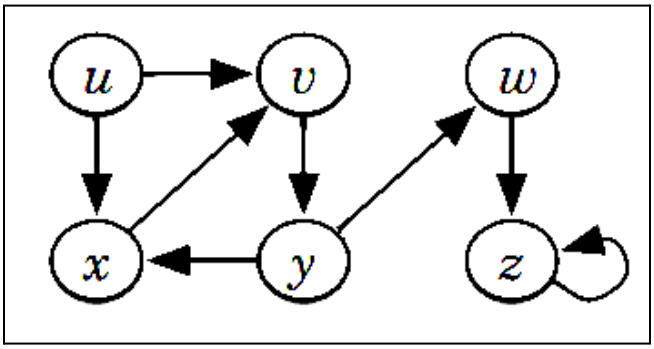
Initial Graph:



DFS (Depth-First Search)

- **Uses a stack** (Last In, First Out) or recursion.
- **Goes deep first**: Start from a vertex, keep going to one neighbor until no more neighbors, then backtrack.
- **Steps**:
    1. Insert the starting vertex into the **stack** and mark it visited.
    2. While the stack is not empty
        - Pop a vertex from the top.
        - Visit its unvisited neighbors and push them onto the stack.
- Good for exploring all paths or detecting cycles.

Initial Graph:




```
G = Adjacency Matrix/List
s = source vertex
visited = [False] * total vertices
queue = new Queue()

BFS(G, s):

  visited[s] = True
  queue.enqueue(s)
  while queue is not empty:
    u = queue.dequeue()
    print(u)
    for each neighbor v of u:
      if visited[v] == False:
        visited[v] = True
        queue.enqueue(v)
```

```
G = Adjacency Matrix/List
s = source vertex
visited = [False] * total vertices
stack = new Stack()

DFS(G, s):

  visited[s] = True
  stack.push(s)
  while stack is not empty:
    u = stack.pop()
    print(u)
    for each neighbor v of u:
      if visited[v] == False:
        visited[v] = True
        stack.push(v)
```

Detailed notes on Graphs:

https://github.com/zabermd/Data-Structures-and-their-Use-in-Elementary-Algorithms/blob/main/Book%20Chapter/Chapter_7.pdf